

LC 1559: Detect Cycles in 2D Grid

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 29, 2026 | Target: Nutanix

Core Concept & Problem Logic

Key Insight:

Whenever DFS reaches a current cell and finds a neighbour already visited, there are **2 possibilities**:

1. That visited = parent

Simply ignore as it doesn't mean a cycle.

2. That visited = not parent

Explored an already seen vertex through another path, **hence cycle**.

State Representation:

Hence, STATE becomes: `dfs(r, c, pr, pc)` (where *pr, pc* = parent cell)

Visual Examples

Example 1: No Cycle (Linear Path)

Path: A — B — C

Analysis:

Neighbour of C = B.

Have we seen B? **Yes**, but it's a parent. So ignore.

Result: NO CYCLE.

Example 2: Cycle Found

Path: A – B – C – D – (back to A)

Analysis:

Neighbour of D = A.

Parent of D is C, and not A. A was seen before.

Result: CYCLE.

C++ Implementation

```
class Solution {
public:

    vector<vector<int>> seen;
    vector<pair<int,int>> dir={{-1,0}, {0,-1}, {1,0}, {0,1}};

    bool dfs(int r, int c, int pr, int pc, vector<vector<char>>& grid) {

        if(r<0 || c<0 || r>=n || c>=m) return false;
        seen[r][c]=true;

        for(auto &it:dir) {
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m) {

                if(grid[nr][nc]==grid[r][c]) {

                    if(seen[nr][nc]==1) {
                        if(nr==pr && nc==pc) continue; // parent he to
koi baat ni
                    } else { // otherwise cycle hoga confirm
                        return true;
                    }
                }

                else
                { // nahi dekh rkha , to aagee traverse krke dekh lo
                    if(dfs(nr,nc,r,c,grid)) return true;
                }
            }
        }
    }
};
```

```

        }
    }
}

return false;
}

int n;
int m;

bool containsCycle(vector<vector<char>>& grid) {

    // basic idea ye he ki
    // same same value cell ko traverse krte rhenge jab tak koi
    visited na mil jaaye
    // agar koi already visited mila
    // to ya to wo parent he ya to cycle aa gyi

    n=grid.size();
    m=grid[0].size();
    seen.assign(n,vector<int>(m,0));

    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            // dfs state me parent bhi rkhna padega because of 1st
            case
            if(!seen[i][j]) {
                if(dfs(i,j,-1,-1,grid)) {
                    return true;
                }
            }
        }
    }

    return false;
}
};

```